

MIE1517 Sample Midterm

Instructor: Sinisa Colic

Date: N/A

Duration: 90 Minutes

Aids Allowed: None

Student Number: Solutions
Family Name(s): _____
Given Names(s): _____

*Do **not** turn this page until you have received the signal to start.
In the meantime, please read the instructions below carefully.*

This test consists of 8 questions on 15 pages (including this one), printed on both sides of the paper. *When you receive the signal to start, please make sure that your copy of the test is complete, fill in the identification section above, and write your name on the back of the last page.*

Answer each question directly on the test paper, in the space provided. If you need more space for one of your solutions, use the extra pages at the end of the test paper and *indicate clearly the part of your work that should be marked.*

Write up your solutions carefully! If you are giving only one part of an answer, indicate clearly what you are doing. Part marks might be given for incomplete solutions where it is clearly indicated what parts are missing.

You **must** write the test in pen if you would like to potentially request for the test to be regraded.

MARKING GUIDE

1: ____/10

2: ____/4

3: ____/7

4: ____/10

5: ____/10

6: ____/8

7: ____/6

8: ____/10

TOTAL: ____/65

Good Luck!

Question 1. [10 MARKS]

Circle the best answer for each of the questions below.

Part (a) [1 MARK]

Which of the following neural network layer has the largest number of parameters?

- (A) `nn.Dropout(p=0.5)`
- (B) `nn.Linear(in_features=3, out_features=3)`
- (C) `nn.Conv2d(in_channels=1, out_channels=1, kernel_size=3).`
- (D) `nn.Conv2d(in_channels=3, out_channels=3, kernel_size=1)`
- ☒ (E) Both (b) and (d) have the largest number of parameters.

Part (b) [1 MARK]

Suppose that we have 100,000 labelled examples in our training set. If our batch size is 100, how many iterations are there in one epoch?

- (A) 10
- (B) 100
- ☒ (C) 1000
- (D) 10,000
- (E) 100,000

Part (c) [1 MARK]

Multilayer perceptron is an example of ...

- (A) A way to prevent overfitting.
- (B) An optimizer.
- (C) A loss function.
- (D) A data augmentation technique.
- ☒ (E) A neural network architecture.

Part (d) [1 MARK]

Can you name the main innovation in AlexNet?

- (A) Skip connections
- ☒ (B) Deeper networks
- (C) Transfer learning
- (D) Inception modules
- (E) Both (b) and (c).

Part (e) [1 MARK]

How many multiplications are performed when applying a 2x2 convolution on a colour image of size 4x4, with stride 1 and no padding?

- (A) 243
- (B) 225
- (C) 81
- (D) Convolution cannot be done
- ☒ (E) None of the above

Good Luck!

Part (f) [1 MARK]

You are training a model to determine whether a picture contains one of eight hand gestures. Which loss function should you use?

- (A) `optim.SGD`
- (B) `nn.BCEWithLogitsLoss`
- (C) `nn.MSELoss`
- ☒ (D) `nn.CrossEntropyLoss`
- (E) Both (a) and (d)

Part (g) [1 MARK]

What type of Machine Learning algorithm would you use to allow a robot to walk in various unknown terrains?

- (A) Unsupervised Learning.
- (B) Supervised Learning.
- (C) Semi-supervised Learning.
- ☒ (D) Reinforcement Learning.
- (E) Competitive Learning.

Part (h) [1 MARK]

The backward pass updates the following:

- ☒ (A) The parameters only.
- (B) The layers only.
- (C) The activations only.
- (D) The layers and activations.
- (E) The parameters and activations.

Part (i) [1 MARK]

A 2D convolutional layer with a 1x1 kernel size ...

- (A) is often used as the first layer of an image autoencoder.
- (B) is computationally more expensive than a 3x3 kernel size.
- (C) has exactly 1 weight and 1 bias (2 parameters).
- ☒ (D) is often used in a GoogLeNet.
- (E) does not make sense.

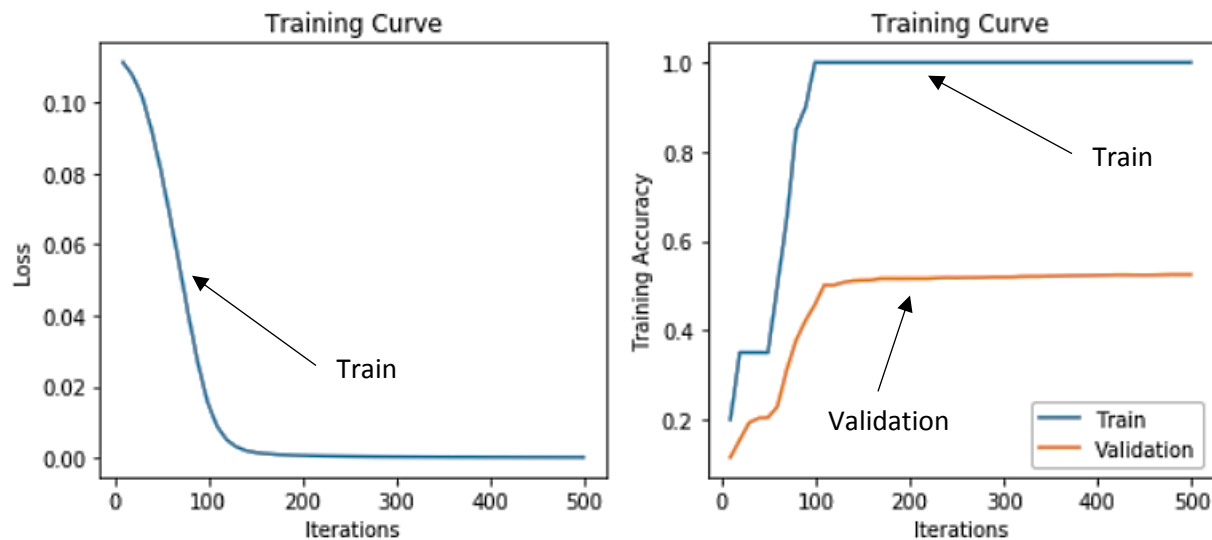
Part (j) [1 MARK]

The setting `model.eval()` is used with which of the following techniques to prevent overfitting?

- (A) weight decay
- ☒ (B) dropout
- (C) data normalization
- (D) data augmentation
- (E) Both (b) and (c)

Question 2. [4 MARKS]**Part (a)** [2 MARKS]

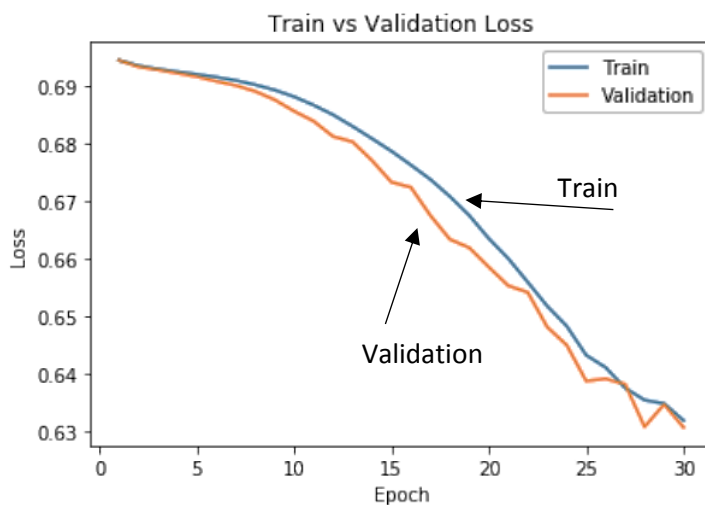
Explain what could cause the training curve to have the below shape.



This curve resembled overfitting on a small training set. The data has overfit the training data which does not capture the properties of the validation dataset.

Part (b) [2 MARKS]

Explain what could cause the training curve to have the below shape.



The model has not settled and should be trained longer. This is an example of underfitting.

Good Luck!

Question 3. [7 MARKS]

Consider ModelA vs ModelB, both of which can be trained to solve binary classification problems. The only difference between the two models is the number of out_features in layer2.

```
class ModelA(nn.Module):
    def __init__(self, input_size, hidden_size):
        super(ModelA, self).__init__()
        self.layer1 = nn.Linear(in_features=input_size, out_features=hidden_size)
        self.layer2 = nn.Linear(in_features=hidden_size, out_features=1)

    def forward(self, input):
        return self.layer2(torch.relu(self.layer1(input)))

class ModelB(nn.Module):
    def __init__(self, input_size, hidden_size):
        super(ModelB, self).__init__()
        self.layer1 = nn.Linear(in_features=input_size, out_features=hidden_size)
        self.layer2 = nn.Linear(in_features=hidden_size, out_features=2)

    def forward(self, input):
        return self.layer2(torch.relu(self.layer1(input)))
```

Part (a) [3 marks]

Assuming the same input_size and hidden_size settings, which model is more likely to overfit? Justify your answer quantitatively.

Model B is more likely to overfit because it has two output neurons compared to only one in Model A. More parameters increases the likelihood of overfitting to the training data.

Part (b) [2 marks]

How will the training code for these two models differ?

The loss function for ModelA will be a binary cross-entropy loss (BCEWithLogitsLoss)
The loss function for ModelB will be a general cross-entropy loss (CrossEntropyLoss)

Part (c) [2 marks]

How will the code to extract prediction probabilities from these two models differ?

ModelA will use a sigmoid to extract the probability for the one output node
ModelB will require a softmax to extract the probability for the two output nodes

Good Luck!

Question 4. [10 MARKS]

What is the expected output of the commands below? You may assume that all required packages like torch and torch.nn are imported. If you expect an error would occur, write “ERROR” without further explanation.

```
>>> x1 = torch.randn(10, 8)
>>> print(x1.shape)
torch.Size([10, 8])

>>> x2 = torch.randn(20, 3, 8, 8) # NCHW
>>> print(x2.shape)
torch.Size([20, 3, 8, 8])
```

Part (a) [2 marks]

```
>>> layer = nn.Linear(in_features=8, out_features=16)
>>> print(layer(x1).shape)
>>> print(layer(x2).shape)
```

```
torch.Size([10, 16])
torch.Size([20, 3, 8, 16])
```

Part (b) [2 marks]

```
>>> conv = nn.Conv2d(in_channels=3, out_channels=7,
    kernel_size=5,
    padding=0)
>>> print(conv(x2).shape)
```

```
torch.Size([20, 7, 4, 4])
```

Part (c) [2 marks]

```
>>> conv2 = nn.Conv2d(in_channels=3, out_channels=7,
    kernel_size=5,
    padding=2)
>>> print(conv2(x2).shape)
```

```
torch.Size([20, 7, 8, 8])
```

Part (d) [2 marks]

```
>>> pool = nn.MaxPool2d(kernel_size=2, stride=2)
>>> print(pool(x2).shape)
```

```
torch.Size([20, 3, 4, 4])
```

Part (e) [2 marks]

```
>>> convt = nn.ConvTranspose2d(in_channels=3, out_channels=1,
    kernel_size=2, stride=2, padding=2)
>>> print(convt(x2).shape)
```

```
torch.Size([20, 1, 12, 12])
```

Question 5. [10 MARKS]

Consider the following incorrect implementation of an autoencoder, to be trained to generate MNIST images.

```
class Autoencoder(nn.Module):
    def __init__(self):
        super(Autoencoder, self).__init__()

        self.encoder = nn.Sequential(
            nn.Conv2d(in_channels=1, out_channels=16, kernel_size=3, stride=2, padding=1),
            nn.ReLU(),
            nn.Conv2d(in_channels=16, out_channels=32, kernel_size=3, stride=2, padding=1),
            nn.ReLU(),
            nn.Conv2d(in_channels=32, out_channels=64, kernel_size=7, padding=0)
        )

        self.decoder = nn.Sequential(
            nn.ConvTranspose2d(in_channels=64, out_channels=32, kernel_size=7, padding=0),
            nn.ReLU(),
            nn.ConvTranspose2d(in_channels=32, out_channels=16, kernel_size=3,
                              stride=2, padding=1, output_padding=1),
            nn.ReLU(),
            nn.ConvTranspose2d(in_channels=16, out_channels=3, kernel_size=3,
                              stride=2, padding=1, output_padding=1),
            nn.ReLU()
        )

    def forward(self, x):
        return self.encoder(self.decoder(x))
```

Part (a) [3 marks]

There are three major issues with the above code. What are they?

1. last convTranspose2d layer should have out_channels = 1 (or first conv layer should have in_channels = 3), both are acceptable
2. final layer should be a sigmoid to normalize the output 0-1
3. in the forward method: the call on encoder and decoder are reversed

Good Luck!

Part (b) [3 marks]

How many total trained parameters are there in the encoder? For this part you are only required to show your work, you are not required to compute the final result.

Encoder Total Parameters:

conv layer 1: $1 \times 3 \times 3 \times 16 + 16$

conv layer 2: $16 \times 3 \times 3 \times 32 + 32$

conv layer 3: $32 \times 7 \times 7 \times 64 + 64$

Part (c) [2 mark]

Alice built an autoencoder to generate images of faces. Unfortunately, she only saved the parameters of the *encoder*, and not the *decoder*. Bailey suggests that Alice only needs to train a new decoder, while keeping the encoder weights fixed to their pre-trained values. Will Bailey's idea work?

Yes, the encoder has been trained to embed the data for reconstruction, hence using the same embedding space as input we should be able to learn a decoder to reconstruct the images.

Part (d) [2 marks]

Christine also wants to build an autoencoder to generate images of faces. She doesn't have a pre-trained encoder like Alice, but decides that she would use a pre-trained AlexNet features network as her encoder. She trains a new decoder, while keeping the encoder weights fixed. Will Christine's idea work? Explain why or why not.

No, the AlexNet has picked up features that are useful for making classifications and these same features may not have enough information about the image to reconstruct it.

Good Luck!

Question 6. [8 MARKS]

Provided below is a confusion matrix applied to a test dataset for a three-class model used to diagnose patients that are healthy (H), have major depression disorder (MDD) or schizophrenia (SCZ).

		Predicted		
		MDD	SCZ	H
Actual	MDD	20	14	16
	SCZ	5	45	0
	H	18	12	70

Part (a) [2 marks]

How are the samples distributed? Is this a balanced dataset?

MDD = 50

SCZ = 50

H = 100

This is not a balanced dataset

Part (b) [2 marks]

For which diagnosis did the model achieve the best prediction result? How many were correctly identified?

Diagnosis: SCZ Correct: 90%

Part (c) [2 marks]

For which diagnosis did the model achieve the worst prediction result? How many were correctly identified?

Diagnosis: MDD Correct: 40%

Part (d) [2 marks]

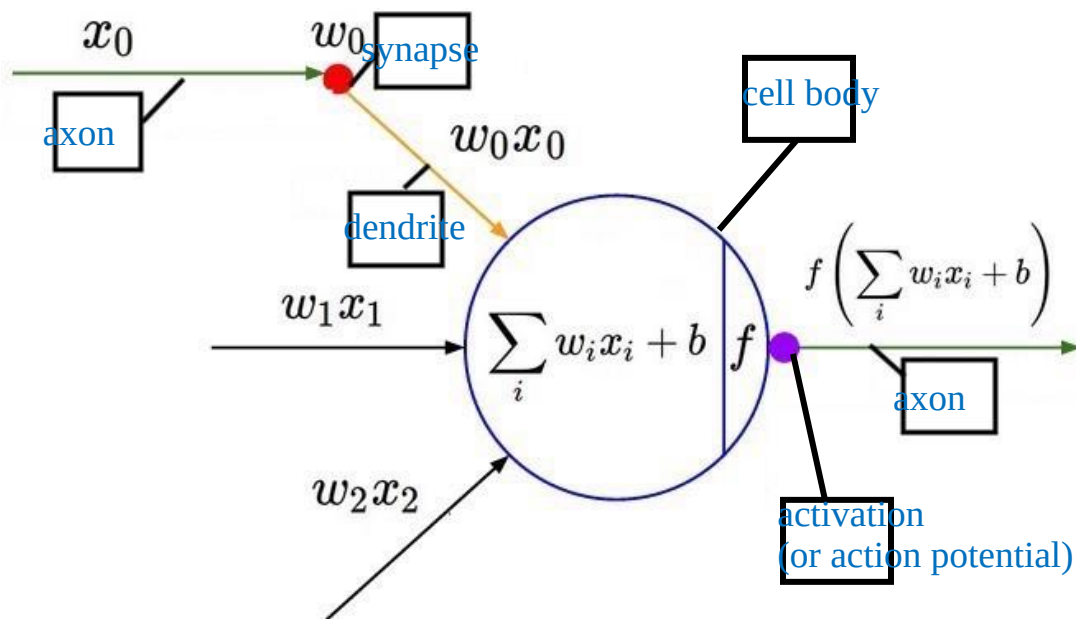
Calculate the total accuracy of the model?

Model Accuracy: 67.5% = correctly predicted / all predictions
= (20 + 45 + 70)/200

Good Luck!

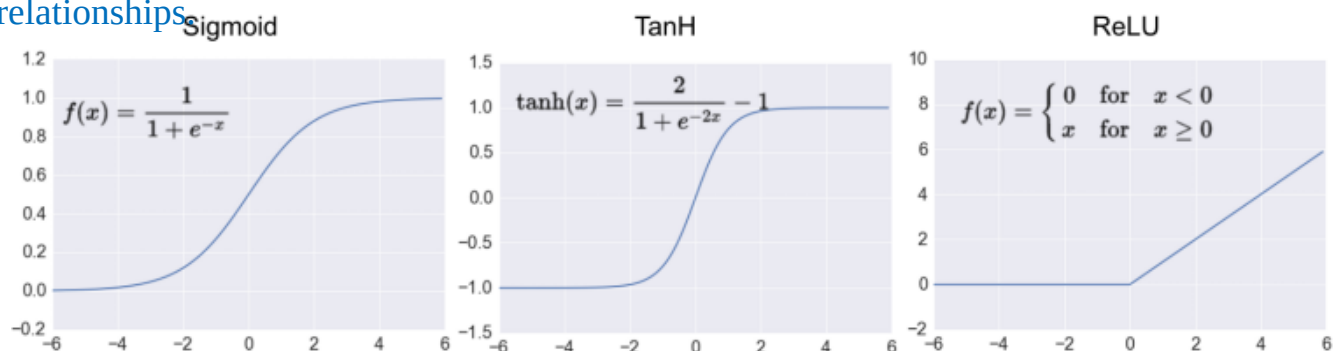
Question 7. [6 MARKS]**Part (a)** [3 MARKS]

Given the drawing of a neuron, identify the following: axon, dendrite, cell body, activation (or action potential) and synapse. You can reuse the terms more than once.

**Part (b)** [3 MARKS]

What is the main reason for using activation functions. Identify and sketch three activation functions introduced in the course.

Introduces nonlinearity into the model. Without it the model cannot capture nonlinear relationships.



Note: Equations are not required for full marks

Good Luck!

Question 8. [10 MARKS]**Part (a)** [2 MARKS]

What is the difference between a model's parameters and hyperparameters?

parameters are updated by the optimizer during training (typically using forms of gradient descent), whereas hyperparameters cannot be learned by the optimizer

Part (b) [2 MARKS]

What happens to the training curve (i.e. training and validation) as you search over more and more combinations of hyperparameters?

As you train over more and more permutations of hyperparameters and updating the hyperparameter selection based on the validation dataset, you are effectively training on the validation set, and hence the model will be fitting both. Over time the difference between the training and validation curves will be minimized assuming a model of sufficient complexity.

Part (c) [2 MARKS]

What is a test set and why would you want to use it? How do you ensure that you use it correctly?

The test set should model what would happen if you were to obtain a new sample. Ideally, the test set should be used only once so that it does not influence the design of the model.

Good Luck!

Part (d) [2 MARKS]

Suppose you use Batch Gradient Descent and you plot the validation error at every epoch. If you notice that the validation error consistently goes up, what is likely going on and how can you correct it?

You are likely overfitting the model to the training data.

This can be corrected by saving the parameters or stopping the training at the point where validation error consistently increases.

Part (e) [2 MARKS]

Which Gradient Descent algorithm (among those we discussed) will reach the vicinity of the optimal solution the fastest?

Adam optimizer

Additional page for answers

Good Luck!

Additional page for answers

Good Luck!

On this page, please write nothing except your name.

Family Name(s): _____

Given Names(s): _____

Good Luck!